

A Neural Network for the Identical Kuramoto Equation: Architectural Considerations and Performance Evaluation

1st Nishantak Panigrahi

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT)
Vellore, India
nishantak09@gmail.com,
nishantak.panigrahi2022@vitstudent.ac.in

2nd Mayank Patwal

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT)
Vellore, India
mayanksingh4370@gmail.com,
mayanksingh.patwal2022@vitstudent.ac.in

Abstract—In this paper, we investigate the efficiency of Deep Neural Networks (DNNs) to approximate the solution of a nonlocal conservation law derived from the identical-oscillator Kuramoto model, focusing on the evaluation of an architectural choice and its impact on solution accuracy based on the energy norm and computation time. Through systematic experimentation, we demonstrate that network configuration parameters—specifically, activation function selection (tanh vs. sin vs. ReLU), network depth (4-8 hidden layers), width (64-256 neurons), and training methodology (collocation points, epoch count)—significantly influence convergence characteristics. We observe that tanh activation yields stable convergence across configurations, whereas sine activation can attain marginally lower errors and training times in isolated cases, but occasionally produce nonphysical artefacts. Our comparative analysis with traditional numerical methods shows that optimally configured DNNs offer competitive accuracy with notably different computational trade-offs. Furthermore, we identify fundamental limitations of standard feed-forward architectures when handling singular or piecewise-constant solutions, providing empirical evidence that such networks inherently oversmooth sharp features due to the natural function space limitations of standard activation functions. This work contributes to the growing body of research on neural network-based scientific computing by providing practitioners with empirical guidelines for DNN implementation while illuminating fundamental theoretical constraints that must be overcome to expand their applicability to more challenging physical systems with discontinuities.

Index Terms—Deep Neural Network Architecture, Activation Functions, Physics-Informed Neural Network, Hyperparameter Tuning, Scientific Computing, Nonlocal Conservation Law, Kuramoto Model, Convergence Analysis, Energy Norm

I. INTRODUCTION

Partial Differential Equations (PDEs) are ubiquitous in modelling processes and their properties arising from a vast range of diverse fields, such as real-time engineering problems, different physical phenomena, chemical and biological events, and socio-economic theory. Efficient and accurate solution of such complex dynamical systems becomes critical across a wide range of these applications. Classical numerical methods, such as finite-volume or finite-difference schemes can deliver accuracy but at the cost of fine spatial meshes,

small time steps, and lengthy runtimes. This computational bottleneck limits their use in scenarios where hundreds or thousands of model evaluations are run in a pipeline or under tight time constraints, especially for high-resolution simulations. Deep neural networks (DNNs) offer a mesh-free alternative by learning a continuous solution directly from the data known about the process, for example, the PDE. We enhance the DNN by informing it about the physics that governs the process, thereby embedding the PDE residual and boundary/initial-condition constraints into the network's loss function, a paradigm henceforth referred to as a Physics-Informed Neural Network (PINN). Consequently, we obtain a model that, once trained, produces solutions to the PDE almost instantaneously. Furthermore, PINNs generalise better beyond the training domain than traditional DNNs due to their incorporation of the governing physical laws directly into the loss function, thereby enforcing strong regularisation and consistency with known physics. Moreover, PINNs often exhibit faster training rates compared to DNNs, as the inclusion of physics-based residuals accelerates convergence by providing structured gradients and reducing reliance on large datasets. This makes PINNs particularly attractive for rapid computation tasks in mission-critical workflows, with early applications to fluid mechanics, wave propagation, and elasticity demonstrating competitive accuracy, and flexibility.

Despite these advances, the use of PINNs for nonlocal conservation laws, particularly those derived from the Kuramoto model, remains underexplored. Moreover, deploying PINNs in real-world engineering requires a clear understanding of how architectural choices (e.g., activation functions, network depth/width) and training strategies (e.g., collocation sampling, epoch budgets) affect the solution fidelity. Furthermore, sharp or piecewise-constant solutions common in synchronisation processes can be poorly captured by smooth activation functions, leading to oversmoothing and nonphysical artifacts. To date, there has been no systematic study quantifying how architectural choices, and training hyperparameters impact the solution accuracy (in an energy or L_2 norm) and computational

efficiency for this class of problems.

In this paper, we fill this gap by conducting a comprehensive evaluation of DNN architectures by approximating the solution to the identical-oscillator Kuramoto Equation. Our contributions are threefold:

- **Architectural analysis:** We compare $\tanh$, $\sin$, and $ReLU$ activations across network depths (4-8 layers) and widths (64-256 neurons), quantifying their influence on convergence behaviour and error in the energy norm. Highlighting cases where sine activations can yield marginal error reductions at the cost of occasional non-physical oscillations, and showcasing the failure of $ReLU$ activation in capturing the evolution of the Kuramoto model.
- **Hyperparameter study:** Through systematic variation of collocation-point sampling and epoch count, we identify configurations that optimise the trade-off between accuracy and training time.
- **Limitations and guidelines:** We demonstrate intrinsic oversmoothing of discontinuities by standard feed-forward DNNs and offer practical recommendations and empirical guidelines for practitioners.

The structure of the remainder of this paper is as follows. Section II introduces the foundational preliminaries, including neural networks, nonlocal conservation laws, and the Kuramoto model. Section III formulates the corresponding PINN approach. Section IV details our experimental setup and methodology. Section V presents results, comparisons, and limitations. Section VI discusses practical considerations, guidelines and possible remedies to fix the limitations. Finally, Section VII concludes the paper.

II. PRELIMINARIES

A. Neural Networks as Function Approximators

A deep neural network (DNN) is a parameterised function $u_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$, defined as a composition of affine maps and nonlinear activations. Effectively, it learns a mapping from an input $x \in \mathbb{R}^d$ (e.g., spatial and temporal coordinates), to an output $f(x) \in \mathbb{R}$.

A typical network is made up of an input layer, several hidden layers, and an output layer. Nonlinear functions, called activation functions (e.g., $\tanh$, $ReLU$), are applied at each neuron, enabling the network to capture complex, nonlinear relationships. When the output of the activation function of the layer is in a specific range only then is that neuron's contribution to the solution is taken into consideration. The network learns by adjusting the network parameters to minimise a loss function.

A Neural Network, $\mathcal{N}(x; \theta)$, looks like

$$\begin{aligned} \mathbf{h}^{(0)} &= x, \\ \mathbf{h}^{(i)} &= \sigma(W^{(i)} \mathbf{h}^{(i-1)} + \mathbf{b}^{(i)}), \quad i = 1, \dots, L, \\ \mathcal{N}(x; \theta) &= W^{(L+1)} \mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}, \end{aligned}$$

where for layer i with n_i neurons in a neural network with L hidden layers,

- $W^{(i)} \in \mathbb{R}^{n_i \times n_{i-1}}$ is the weight matrix,
- $\mathbf{b}^{(i)} \in \mathbb{R}^{n_i}$ is bias matrix,
- σ is nonlinear activation function (e.g., $\tanh$, $\sin$, $ReLU$)

The network parameters, $\theta = \{W^{(i)}, \mathbf{b}^{(i)}\}_{i=1}^{L+1}$, are learnt by minimising a scalar loss $L(\theta)$ typically via gradient-based optimisers (e.g., Adam).

Loss Function: In conventional supervised learning, a neural network is trained by minimising a loss that quantifies the difference between the network's prediction and the true data. For example, given input-output pairs $\{(x_i, y_i)\}$, one could minimise a mean-squared error

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N |\mathcal{N}(x_i; \theta) - y_i|^2.$$

However, when solving a PDE, the “true” solution is usually unknown apriori, otherwise most of humanity's efforts would be moot. Instead, in a physics-informed approach the loss is constructed from the governing equations and any available boundary or initial data.

B. Nonlocal Conservation Laws

A conservation law asserts that all change in a physical quantity, u , in a system (or a domain, Ω) must be due the net change in the flux, $\mathbf{F}$, through that system. This can be expressed by

$$\frac{\partial u(x, t)}{\partial t} + \nabla \cdot \mathbf{F}(x, t) = 0,$$

This assumes that all interaction of the flux is local and happens through an infinitesimal surface around x , i.e., interactions between only the *immediately neighbouring points* in an *infinitely small shell* around x .

A non-local conservation law, however, accounts for interactions that affect $u(x, t)$ outside of this shell. The nonlocal flux will then include the pairwise exchanges of the conserved field $u(x, t)$ between each point $x \in \Omega$ and all points $y \notin \Omega$ that lie within a finite horizon $\delta > 0$. The finite-horizon interaction domain can then be defined as

$$\{(x, y) : x \in \Omega, y \in \Omega^c, \|y - x\| \leq \delta\}.$$

Each of these pairs contribute an interaction

$$K(x, y)[u(y) - u(x)]$$

where $K(x, y)$ is the exchange kernel.

Integrating these interactions over the domain we get a nonlocal flux

$$\mathbf{J}_{nl}(x, t) = \int_{\Omega^c} K(x, y)[u(y, t) - u(x, t)] dV_y$$

The integrand, $K(x, y)[u(y) - u(x)]$, is positive when y donates to x and negative when x loses to y . Our conservation law has thusly become a nonlocal conservation law

$$\frac{\partial u(x, t)}{\partial t} + \nabla \cdot \mathbf{F}(x, t) + \mathbf{J}_{nl}(x, t) = 0,$$

The exchange kernel K defines the nonlocal interactions. We can recover the local conservation law by setting $K \equiv 0$ or set it accordingly for other types of interaction models.

Conservation laws of this form arise in a large variety of applications making such models relevant in numerous implementations, such as for the simulation of crowd dynamics, traffic flow, microbiology and more.

C. Kuramoto Model

The Kuramoto model is a fundamental framework for studying synchronization phenomena in coupled oscillatory systems. It provides a mathematical descriptor for understanding how a large population of self-sustained oscillators can spontaneously synchronize, despite the fact that each one has its own natural frequency.

Imagine N metronomes, each ticking at its own pace, ω_i . All of them are mounted on a common plank. Through this coupling they all *feel* the rhythm of the group and adjust their phases, $\theta(t)$. As the coupling strength, K , increases past a critical threshold, the ensemble undergoes a transition from incoherent (unsynchronised) motion to a macroscopic coherent state. The Kuramoto model reads

$$\frac{d\theta_i}{dt} = \omega_i + \frac{K}{N} \sum_{j=1}^N \sin(\theta_j - \theta_i), \quad i = 1, \dots, N,$$

Physically, each oscillator i tends to drift at its natural rate ω_i but is *pulled* toward the average phase of all others via the *sine*-interaction.

At continuum this density can be tracked by the PDE described in [?]

$$\frac{\partial f}{\partial t}(\theta, \Omega, t) + \frac{\partial}{\partial \theta} \left(v[f](\theta, \Omega, t) \underbrace{f(\theta, \Omega, t)}_{\text{density}} \right) = 0.$$

we write the phase-space flux as

$$J(\theta, \Omega, t) = v[f](\theta, \Omega, t) \times f(\theta, \Omega, t).$$

where $v[f](\theta, \Omega, t)$ is the velocity (angular speed) at which these oscillators carry their mass in the θ direction.

The nonlocality in the Kuramoto equation is introduced through the velocity field [?]

$$v[f](\theta, \Omega, t) = \Omega - K \int_0^{2\pi} \sin(\theta - \theta^*) \rho(\theta^*, t) d\theta^*.$$

We can write the nonlocal term as a convolution with a kernel W as

$$v[f](\theta, t) = \Omega - K \int_0^{2\pi} W(\theta, \theta^*) \rho(\theta^*, t) d\theta^*,$$

where

$$W(\theta, \theta^*) = \sin(\theta - \theta^*) \neq 0; \quad \theta^* \neq \theta.$$

Note that $W(\theta, \theta^*)$ is antisymmetric in its arguments, reflecting that phase pulls are equal and opposite.

The nonlocality comes from this integral which means the angular velocity at phase θ *feels* the contributions from

oscillators at every other phase θ^* . This is an all-to-all mean-field coupling, meaning that the interaction domain, $\Omega \times \Omega^c = S \times S$, where $\theta \in S$.

III. PHYSICS-INFORMED NEURAL NETWORK FORMULATION

A. Identical Kuramoto Equation

The identical-oscillator case sets all $\omega_i = \Omega_0$. Then $f(\theta, \Omega, t) = \rho(\theta, t) \delta(\Omega - \Omega_0)$. Integrating out Ω gives us

$$\partial_t \rho(\theta, t) + \partial_\theta (L[\rho](\theta, t) \rho(\theta, t)) = 0,$$

with $L[\rho](\theta, t) = -K \int \sin(\theta - \theta^*) \rho(\theta^*, t) d\theta^*$.

We approximate the solution to the identical Kuramoto equation by approximating the phase density $u(\theta, t)$, satisfying the PDE

$$\frac{\partial u}{\partial t}(\theta, t) + \frac{\partial}{\partial \theta} [V[u](\theta, t) u(\theta, t)] = 0, \quad \theta \in [0, 2\pi], \quad t \in [0, T],$$

with smooth, polynomial initial condition

$$u(\theta, 0) = u_0(\theta), \quad \theta \in [0, 2\pi].$$

$$u_0(\theta) = \begin{cases} \frac{6}{\pi^3} \left(\frac{3\pi}{2} - \theta \right) \left(\theta - \frac{\pi}{2} \right), & \text{if } \theta \in \left[\frac{\pi}{2}, \frac{3\pi}{2} \right], \\ 0, & \text{otherwise.} \end{cases}$$

The nonlocal flux is given by the convolution

$$V[u](\theta, t) = -K \int_0^{2\pi} \sin(\theta - \phi) u(\phi, t) d\phi,$$

For our simulation we have set the final time, $T = 1$.

B. Network Architecture

The PINN $\mathcal{N}_\Phi$ is constructed as a fully-connected feed-forward network, as shown in Fig. 1, with an input layer of dimension 2 (receiving spatial and temporal coordinates), L hidden layers, each with n neurons, with nonlinear activation functions σ , and a linear output layer that yields the approximation $u(\theta, t)$,

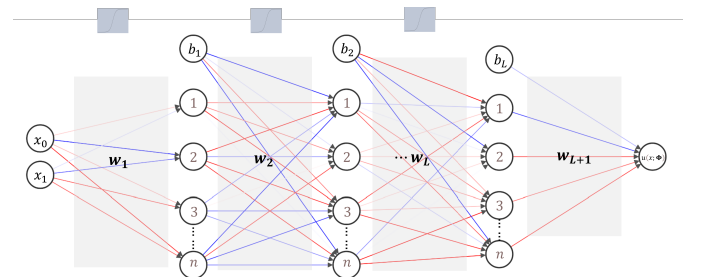


Fig. 1: Network architecture.

It is formulated as

$$\mathcal{N}(x; \Phi) : \mathbb{R}^2 \rightarrow \mathbb{R}$$

For our architecture we have chosen $L=4$, $n=64$, and $\sigma=\tanh$. This gives us

$$\begin{aligned} x &= [\theta, t]^\top, \\ \mathbf{h}^{(0)} &= x, \\ \mathbf{h}^{(i)} &= \tanh(W^{(i)} \mathbf{h}^{(i-1)} + \mathbf{b}^{(i)}), \quad i = 1, \dots, 4, \\ \mathcal{N}(x; \Phi) &= W^5 \mathbf{h}^{(4)} + \mathbf{b}^{(5)} = u(\theta, t) \end{aligned}$$

The trainable parameters are $\Phi = \{W^{(i)}, \mathbf{b}^{(i)}\}_{i=1}^5$.

C. Loss Components

1) *PDE Residual*: Defines the point-wise residual of our PDE

$$r(\theta, t; \Phi) = \frac{\partial u_\Phi}{\partial t} + \frac{\partial}{\partial \theta}(V[u_\Phi] u_\Phi)$$

Sampling N_r collocation points $\{(\theta_i, t_i)\}$, we form

$$L_{\text{res}} = \frac{1}{N_r} \sum_{i=1}^{N_r} r(\theta_i, t_i; \Phi)^2.$$

Nonlocal Flux: Nonlocal convolution, $V[u]$ is computed via a Riemann sum. We introduce $N_q=128$ uniformly spaced nodes $\{\phi_j\}_{j=1}^{N_q}$ on $[0, 2\pi]$ with step size $\Delta\phi = 2\pi/N_q$. The integral is then approximated as

$$V[u_\Phi](\theta_i, t_i) \approx -K \sum_{j=1}^{N_q} \sin(\theta_i - \phi_j) u_\Phi(\phi_j, t_i) \Delta\phi,$$

2) *Initial Condition Loss*: We enforce $u_\Phi(\theta, 0) = u_0(\theta)$ by sampling $N_0=512$ initial condition points, $\{\theta_j^0\}_{j=1}^{N_0}$, and computing

$$L_{\text{IC}} = \frac{1}{N_0} \sum_{j=1}^{N_0} |u_\Phi(\theta_j^0, 0) - u_0(\theta_j^0)|^2.$$

3) *Total Loss*: The total physics-informed loss that the network sees becomes a weighted sum

$$L_{\text{total}} = \lambda_{\text{res}} \cdot L_{\text{res}} + \lambda_{\text{IC}} \cdot L_{\text{IC}}.$$

where λ_{res} , λ_{IC} are the weights of the residual loss and the initial condition loss respectively. In our implementation we have set $\alpha = \beta = 1$.

D. Training and Optimisation

- **Automatic Differentiation**: All derivatives $\partial_t u_\Phi$ and $\partial_\theta(\mathbf{F}_\Phi)$ are computed via the framework's auto-grad engine, which is significantly faster than numerical differentiation and one of the motivations behind using neural networks for scientific computing.
- **Sampling Strategy**: 1024 collocation points (θ_i, t_i) are drawn uniformly via Latin-Hypercube sampling over $[0, 2\pi] \times [0, T]$. 512 initial-condition points θ_j^0 are uniform over $[0, 2\pi]$.
- **Optimiser**: Parameters are updated using gradient-based backpropagation optimiser - Adam, with learning rate 10^{-3} , typically over 4096 or 5120 epochs.

IV. EXPERIMENTAL SETUP & METHODOLOGY

A. Network Architecture and Hyperparameter Grid

We train and evaluate fully-connected feed-forward DNNs by systematically varying the architecture as follows:

- Activation function, $\sigma \in \{\tanh, \text{sine}, \text{ReLU}\}$
- Network depth, L , and width, n , with

$$\{L, n\} \in \{ \{4, 64\}, \{4, 128\}, \{6, 128\}, \{6, 256\}, \{8, 256\} \}$$

We vary training hyperparameters as follows:

- Epoch counts, $n_e \in \{2048, 4096, 5120, 10240\}$
- Collocation points, $N_r \in \{1024, 2048\}$.

Each trained network, $\mathcal{N}_\Phi^{\{\sigma, \{L, n\}, n_e, N_r\}}$, learns the trainable parameters Φ to map an input $x = \{\theta, t\}$ to the output $u_\Phi(\theta, t)$, passing it through L hidden layers of size n , and a linear output layer.

B. Evaluation Metrics

1) *Exact Solution*: We compute a high-fidelity solution, $u_{\text{ex}}(\theta, t)$, using a high-resolution finite-volume solver described in [?].

A super-fine mesh solution with 512 spatial points, and 205 temporal points over the domain, is saved to serve as the reference solution for evaluating and comparing the accuracy of the PINN solutions. The method uses Lax-Friedrichs numerical flux.

The finite-volume scheme

$$\mu_j^{n+1} = \frac{\mu_{j-1}^n + \mu_{j+1}^n}{2} - \frac{\Delta t}{\Delta x} \frac{V[\mu^n]_{j+1} \mu_{j+1}^n - V[\mu^n]_{j-1} \mu_{j-1}^n}{2}$$

can be written as

$$\mu_j^{n+1} = \mu_j^n - \frac{\Delta t}{\Delta x} [F_{j+1/2}^n - F_{j-1/2}^n]$$

where

$$\begin{aligned} F_{j-1/2}^n &= F(\mu_{j-1}^n, \mu_j^n) \\ F_{j+1/2}^n &= F(\mu_j^n, \mu_{j+1}^n) \end{aligned}$$

where Lax-Friedrichs numerical flux

$$F(\mu_j^n, \mu_{j+1}^n) = \frac{f(\mu_j^n) + f(\mu_{j+1}^n)}{2} - \frac{\Delta x}{2\Delta t} [\mu_{j+1}^n - \mu_j^n]$$

where nonlocal flux

$$f(\mu_j^n) = -KV[\mu^n]_j \mu_j^n$$

2) *Energy Norm*: We quantify the solution accuracy by calculating the discrete L^2 norm

$$\|u_\Phi - u_{\text{ex}}\|_E = \sqrt{\frac{1}{N_{\text{eval}}} \sum_{k=1}^{N_{\text{eval}}} (u_\Phi(\theta_k, t_k) - u_{\text{ex}}(\theta_k, t_k))^2}$$

This gives us a single scalar measuring the deviation between the network's prediction and the high-fidelity reference solution. Physically, $\|u_\Phi - u_{\text{ex}}\|_E$ represents the *total energy* of the error field, accumulated over all phases and times, giving us a metric for direct comparison of architectures.

3) *Training Time*: The total wall-clock time each network configuration takes to finish training is measured in *seconds* using Python’s *time* module, and serves as a direct metric of computational efficiency across the grid.

C. Training Hardware

All experiments are conducted on a laptop configured with an AMD Ryzen 7 7735HS CPU (8 cores, 16 threads; 3.2 GHz base, 4.75 GHz boost), 16 GB of system memory, and an NVIDIA GeForce RTX 3050 dGPU (6 GB VRAM) running CUDA 12.8. Neural network training is performed on the dGPU via PyTorch’s CUDA backend, with a fixed seed to ensure consistency and reproducibility of timing measurements.

V. RESULTS AND COMPARISONS

In this section, we present a quantitative evaluation of the trained networks across our architectural and hyperparameter grid, and analyse the effect of varying activation function, network depth and width, collocation density, and epoch budget, on solution accuracy and training efficiency.

A. Activation Functions

Networks with hyperbolic-tangent activation, *tanh*, consistently achieved the most reliable convergence. The best configuration $\sigma = \tanh, \{L=4, n=128\}, N_r=1024, n_e=4096$ showed the least energy norm, $RMSE = 6.65 \times 10^{-5}$ with a training time of ~ 114 *seconds*. The evolution of the solution is shown in Fig. 2. Approximately 5% of *tanh* networks showed nonphysical artefacts. Importantly, these occurrences were confined to configurations with excessive depth and width, suggesting that over-parameterisation for a simple PDE leads the model to fit spurious features rather than generalise the true solution structure.

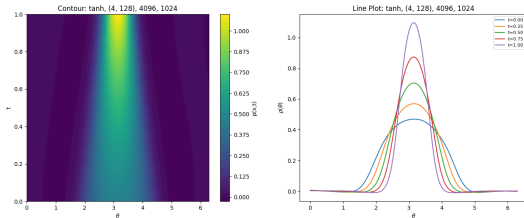


Fig. 2: Solution evolution of *tanh* network.

Best-case trials for *sin* networks occasionally yielded marginally lower energy norm for the same architecture with *tanh* activation, showing approximately 10% lower energy norms, and approximately 13% faster convergence rate than *tanh* architectures with similar energy norm, as shown in Fig. 3. The evolution of the solution is shown in Fig. 4. However, such networks produced nonphysical artefacts in approximately 10% of configurations. These artefacts were, predominantly, spurious high-frequency oscillations which can be attributed to the intrinsic oscillatory nature of the sine activation, whose nondecaying, sign-alternating derivative, cosine,

amplifies unresolved spectral modes under insufficient collocation density, a behaviour not observed with *tanh*, whose monotonic, low-pass characteristic and bounded derivative dampen such perturbations.

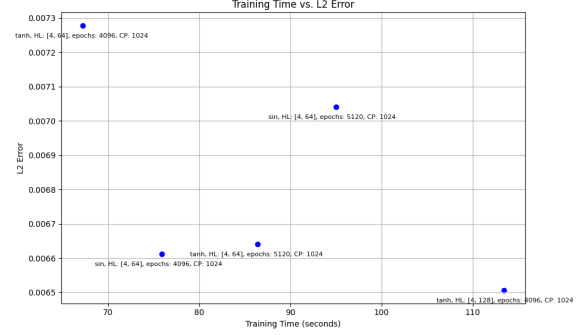


Fig. 3: Training time vs. L2 error of *sin* and *tanh* networks

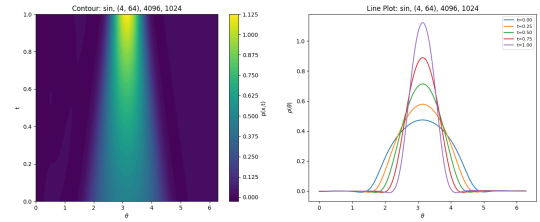


Fig. 4: Solution evolution of *sin* network.

Fig. 5 shows that *ReLU* networks consistently fail to capture the solution across all architectures and hyperparameters. These results confirm that smooth activation functions, such as *tanh*, *sin* are essential to scientific computing using DNNs.

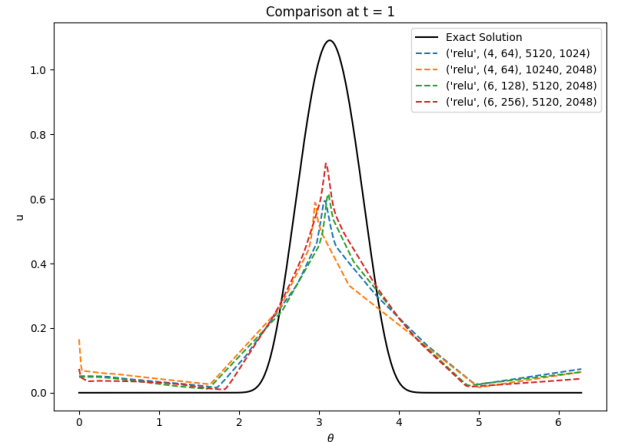


Fig. 5: Solution of *ReLU* networks.

B. Network Depth and Width

We vary network depth and width in a *tanh* network and quantify accuracy and efficiency, as shown in Fig. 7.

The configuration $\{L=4, n=64\}$, $N_r=1024$, $E=2048\}$ completes training in only 35 s but incurs a relatively high error of 1.11×10^{-4} . Doubling the epoch budget to $E = 4096$ for the same architecture yields approximately a twofold reduction in error ($\sim 7 \times 10^{-5}$) for a modest increase in training time (~ 70 s).

Increasing the width from $n=64$ to $n=128$ (with $L=4$, $N_r=1024$, $E=4096$) raises the training time to ~ 114 s but further reduces error down to 6.65×10^{-5} . Demonstrating that in simple PDEs, to capture the solution better, widening may be more effective than simply extending epochs.

Extending epochs, n_e beyond 5120 produces only marginal error improvements but significant increases in training times, indicating that the networks converged in $n_e \leq 5120$.

Deepening to $L=6$ and widening to $n=256$ dramatically increases training time to > 1000 s, while offering negligible error reduction. This confirms that large feed-forward architectures do not offer any benefits over smaller networks, for simple PDEs.

Overall, the Pareto-front of minimal training time versus minimal L^2 error is traced by shallow ($L=4$), moderately wide ($n = 64-128$) networks with epoch counts in the range 4096–5120, giving us the best balance between convergence speed and accuracy. The solutions produced by these architectures are shown in Fig. 6, in contrast to a ReLU-based plot.

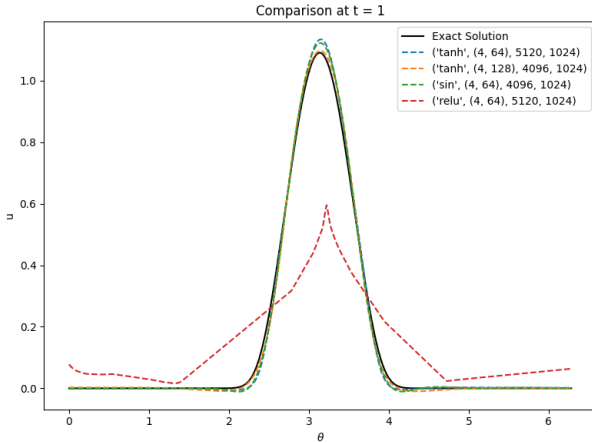


Fig. 6: Solution of trained networks against reference solutions.

C. Collocation Density

Fig. 7 shows that raising the collocation-point count from $N_r=1024$ to $N_r=2048$ does not produce any substantial accuracy gains unless accompanied by either more training epochs or a larger network. Even then the gains are marginal ($<10\%$). This behaviour suggests that denser sampling alone cannot improve the solution unless the network has sufficient representational capacity and training time to exploit the extra information. Empirically, $N_r=1024$ consistently struck a good balance between most of the accuracy benefits while keeping the wall-clock cost low, particularly for simple PDEs, and for

our chosen shallow ($L=4$) architectures with moderate width ($n = 64-128$).

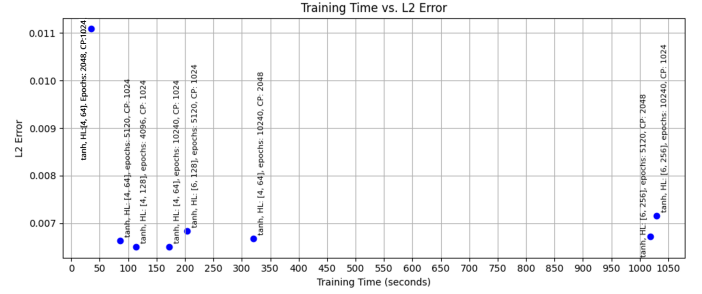


Fig. 7: Training time vs L2 error of tanh networks.

D. Limitations: Discontinuous Initial Data

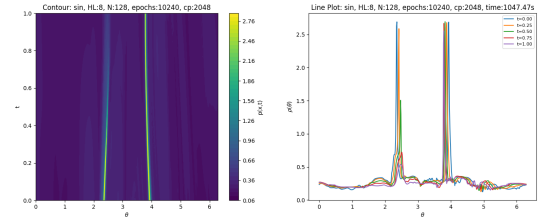
While standard feed-forward PINNs demonstrate competitive accuracy on smooth initial conditions, our experiments reveal fundamental shortcomings when the initial data contains singularities or piecewise-constant functions. We explore these shortcomings by training several architectures and activation functions on

1. Dirac-delta initial condition described in [?]

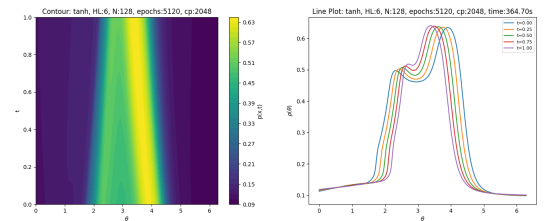
$$\delta_\varepsilon(x-a) = \begin{cases} \frac{1}{2\varepsilon}, & |x-a| < \varepsilon, \\ 0, & \text{otherwise,} \end{cases} \quad H(x) = \begin{cases} 1, & x \geq 0, \\ 0, & x < 0. \end{cases}$$

$$u_0(\theta) = \frac{1}{4} [\delta_\varepsilon(\theta - \frac{3\pi}{4}) + \delta_\varepsilon(\theta - \frac{5\pi}{4})] + \frac{1}{2} H(\theta - \frac{\pi}{2}) [1 - H(\theta - \frac{3\pi}{2})].$$

The closest solutions evolved as shown in Fig. 8.



(a) sin activation



(b) tanh activation

Fig. 8: Closest solution evolution for initial data with singular masses.

2. Piecewise-constant initial condition described in [?]

$$u_0(\theta) = \begin{cases} \frac{2}{3\pi}, & \frac{\pi}{2} \leq \theta \leq \frac{3\pi}{2}, \\ \frac{1}{3\pi}, & \text{otherwise.} \end{cases}$$

The closest solutions evolved as shown in Fig. 9.

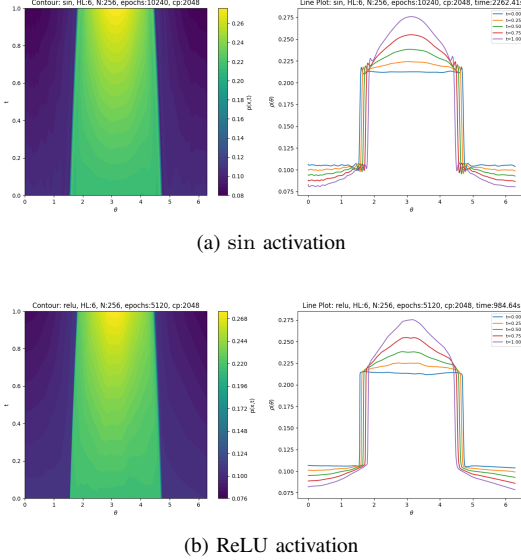


Fig. 9: Closest solution evolution for initial data with discontinuities.

We find that all tested configurations oversmooth the sharp jumps and discontinuities present in these initial conditions. The output approximate solution looks like a smooth function, as shown in Fig. 8(b), dampening these discontinuities. The piecewise-constant data is approximated more accurately than the singular data, by networks using sin and *ReLU* activations, due to the oscillatory behavior of sin and the piecewise-linear nature of *ReLU*. However, as shown in Fig. 9, both networks exhibit spurious oscillations near discontinuities—a Gibbs phenomenon-like artefact, arising from the difficulty of representing abrupt changes by the network.

However, this is consistent with neural-network behaviour because of:

- **Smooth Activation Bias:** Smooth activation functions used in DNNs for scientific computing (e.g., tanh, sin) enforce C^∞ solution spaces. This means that the resulting composite map

$$u_\Phi(\theta, t) = W^{(L+1)} \sigma(W^{(L)}(\dots \sigma(W^{(1)}[\theta, t]) \dots))$$

is in C^∞ class, i.e., an infinitely differentiable function. Therefore, it cannot have a true discontinuity or delta-like spikes. Any such feature would require an unbounded derivative, which at that point would blow up. So, even with very fine collocation grids ($N_r \geq 2048$) and extended training ($n_e \geq 10240$) the optimiser is confined to move within the C^∞ manifold of smooth functions, thereby

smoothing and rounding-off any spikes and jumps in the representation.

- **Strong formulation of PDE:** A standard PINN sees the PDE in its strong (point-wise) form

$$r(\theta_i, t_j) = \partial_t u_\Phi(\theta_i, t_j) + \partial_\theta [V[u_\Phi](\theta_i, t_j) u_\Phi(\theta_i, t_j)],$$

and then trains to minimise $MSE(r(\theta_i, t_j)) \rightarrow 0$. Because $u_\Phi \in C^\infty$, every derivative is also a smooth, finite function. Automatic differentiation using auto-grad gives us the strong derivative of a smooth network even when the true solution has weak derivative or a distributional derivative like a delta function.

At the location of a jump, u_Φ smoothly interpolates the discontinuity. Rounding off this jump changes r only slightly as the network learns to balance the terms, thereby misinterpreting the physics of the discontinuity. Hence, there is no large, localised penalty at that interface that would push the network to *steepen* this curve and *sharpen* the corners. All the optimiser ever *knows* is that its smooth approximation satisfies $r \approx 0$ in the *MSE* and it achieves this most easily by converging to a lower-curvature surrogate that smoothens the discontinuities.

VI. PRACTICAL CONSIDERATIONS

Our experiments and findings clearly lay empirical foundations for architectural considerations in DNN models for scientific computing. Practitioners must heed the following guidelines and, where necessary, adopt specialised remedies to overcome the limitations identified in Section V.

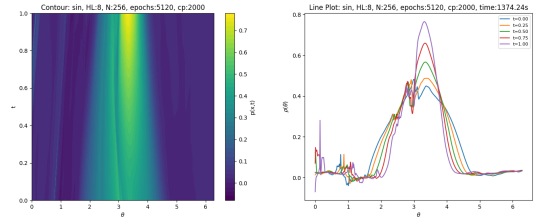


Fig. 10: Nonphysical artefacts in a sin network evolution.

A. Architecture and Training Guidelines

1) **Activation Functions:** tanh proves to be an inherently reliable activation function for smooth function approximators, while sin can introduce intrinsic oscillations and nonphysical artefacts, as shown in Fig. 10.

2) **Depth and Width:** Empirically, simple PDEs can be approximated by shallow, $L=4$ networks. Even with increasing complexity, networks with 4–6 hidden layers, and 64–128 neurons per layer achieve near-optimal accuracy/time trade-offs, as shown in Fig. 7. However, for simple PDEs caution must be exercised as over-parameterization may lead to *memorisation* rather than generalisation. Fig. 7 shows that larger models do not yield noticeable error reduction, while incurring significant additional compute cost.

3) *Collocation Density and Epoch Budget*: As shown in Fig. 7, collocation density of $N_r \approx 1024$ and epochs, $n_e \approx 5120$ consistently provides the most achievable accuracy and training time in typical GPU-laptop runtimes. Scaling either yields negligible gains, and disproportionately larger training times. To avoid wasted computations, it is essential to monitor convergence of the training loss during epochs along with techniques like early-stopping.

B. Remedies for Discontinuities and Spikes

To mitigate the inherent oversmoothing of discontinuities by standard feed-forward DNNs, future studies and applications could look into any of the following techniques: weak or variational loss formulations that integrate the residual against test functions, thereby providing distributional derivatives and gradients at jumps. Domain decomposition by training separate subnetworks on subintervals so as to localize shocks and prevent global smoothening; this would however require apriori knowledge of domain and solution structure. Enriched input embeddings (e.g., Fourier features or wavelets) and adaptive collocation strategies focused on regions of high residual gradient may further enhance the network’s expressivity to capture steep gradients. Physics-based regularisation terms (e.g., mass conservation) can discourage spurious smoothening. One emerging field is quantum machine learning, wherein Parameterised Quantum Circuits (PQCs) use quantum effects to capture nonlocal correlations. While the output of a quantum PQC might still be smooth in Hilbert space, hybrid classical-quantum schemes might combine shock-capturing nonlocal correlations for sharp features, with classical approximators for the smooth domain. Work in these techniques can extend DNNs’ applicability to problems featuring discontinuities and singular measures.

VII. CONCLUSION

Within this work, we have presented a systematic and comprehensive study of feed-forward DNN architectures, training styles, and their inherent limitations. By applying a PINN to a nonlocal conservation law - the identical-oscillator Kuramoto equation, and performing an extensive grid search over activation functions, network depth and width, collocation density, and epoch budgets, we identified that moderately-sized networks with four to six hidden layers of 64 to 128 neurons each, with tanh activation, provide the most competitive accuracy, and training times. Our experiments show that, for smooth initial data, these PINNs achieve an energy norm of the order 10^{-5} within ~ 100 *seconds* of training. With empirical evidence we have thus provided a practical guideline for practitioners to deploy DNNs for scientific computing and industrial applications.

We have empirically shown the limitations of standard DNNs in approximating functions with discontinuities and singular masses due the combined effect of smooth activation function and auto-grad on strong-form PDE residual. To overcome these shortcomings we have outlined possible future directions of research with promising scope.

Therefore, this work effectively enriches the current body of literature on the use of DNNs for scientific and industrial applications.

REFERENCES

Common sense, general knowledge, ask me.